Aran Learn 21: How the Explainer Video Is Made

For: anyone who wants to change a line in the video, fix a picture, or make a new chapter. **Time:** about 1 hour to read and try one small change. **Before this:** Learn 13 (animation and visual explainers).

There are two versions of the video, built the same way:

Version	Folder	Length	Style
Full course (v2)	`artifacts/Explainer video v2/`	76.6 min, 6 parts, 61 chapters	Light "paper" look, characters, moving camera, a recap after every chapter, real incidents
Quick tour (v1)	`artifacts/Explainer video/`	24 min, 22 chapters	Dark look, shorter explanations

Neither is edited by hand in a video editor. Each is built from text files by a few scripts, so anyone on the team can change a sentence and rebuild it the same way every time. This page describes v2; v1 works the same way with fewer helpers.

1. The big picture

```
script.txt  ──tts.py──►  one WAV per line  ──►  narration.wav + timing.json
                                                          │
scenes/<chapter>.js  ◄────────────────────────────────────┘
      │   (drawn with player/engine.js)
render.js: open the player in a headless browser, take one picture per frame
      │
build.py: chunks → one video per part → full video, + narration + subtitles
```

The rule that holds everything together: **the voice decides the timing, and the pictures follow the voice.** Every animation is attached to a spoken line, not to a fixed second.

2. The script: plain words first

`script.txt` is plain text. A line starting with `##` starts a chapter:

```
## checkdigit | Part 4 · Chapter 22 | The last digit is a lock
A check digit is one extra digit computed from all the others.
The subtitle text || The spoken text, when they need to differ
```

- Each line under the header is one spoken sentence and one subtitle. The middle field is the label shown above the chapter title.
- A header can end with `| part` (a part title card) or `| noheader`.
- `text || say` lets the subtitle show "X25519" while the voice says it in a way a listener can follow.

v2 was written for viewers with no technical background, so the script follows three rules:

1. **An everyday example comes before every technical word** (a waiter for an API, a cloakroom ticket for a token, a lock and key for a check digit, a smoke alarm for detection).
2. **Real incidents make the problem concrete**: Hitachi Payment Services (2016), the SBI open server (2019), Juspay (2020), SITA and Air India (2021), the CoWIN Telegram bot (2023), Star Health (2024), and abroad Twitter (2018), Facebook (2019) and Samsung (2023). A later chapter, "What if Aran had been there?", goes back to each case and says honestly what Aran would and would not have changed.
3. **Every chapter ends with "Recap."** and three short points.

3. The voice

`tts.py` uses **Kokoro**, a small open text-to-speech model (82 million parameters) that runs on the CPU through ONNX Runtime. Nothing is sent to an outside service.

- Voice `af_heart`, speed 0.85 (slower than v1, so it is easy to follow), 24 kHz audio, about 132 words a minute.
- A pronunciation table reads names and short forms correctly (Aran → "Uh-run", Verhoeff → "Ver-hoff", OTP → "O T P", ₹250 crore → "250 crore rupees").
- One WAV per line, saved in `audio/` under a name made from a hash of the voice, speed and spoken text. Change one line and only that line is recorded again.
- Silence at the edges of each WAV is trimmed, then fixed gaps are added: 1.0 s before a chapter's first line, 0.8 s between lines, 1.8 s after a chapter (2.4 s after a part title).
- Output: `narration.wav` and `timing.json` / `timing.js` (when each line starts and how long it lasts).

4. The player: a frame is a function of time

`player/player.html` is a 1920×1080 SVG drawing. Calling `seek(t)` draws exactly what the video shows at second `t`, from scratch. It never depends on what was drawn before, so four workers can render different parts at once and a snapshot of any second always looks the same.

The look

All colours come from one palette in `engine.js`: a warm off-white page, dark ink text and six calm colours (blue, red, green, ochre, plum, teal), each with a light tint. There are no gradients, glows or dark backgrounds. Titles use a serif font (Fraunces), body text a sans font (Inter), and numbers a monospace font.

Building a chapter

Each chapter is one file, `scenes/<id>.js`, that builds its pictures once and then says when each one appears, in terms of the spoken lines:

```
scene('checkdigit', X => {
  X.cam([{ x: 960, y: 420, s: 1.12 }, { k: 3, x: 960, y: 560, s: 1 }]); // camera: zoom in, then pull back a
  const row = X.digits(960, 190, '234567890124', { groups: [4, 4, 4] });
  X.show(row, 0, { type: 'up' });                // slides up when line 0 starts
  X.term(2, 'Check digit', 'One extra digit calculated from the others.'); // word card
  X.recap(9, ['The check digit is a lock', 'The other eleven digits are the key', 'One wrong digit: the lock
});
```

- `k` is the line number inside the chapter and `f` is how far through that line (0 to 1).
- **Word cards** (`X.term`) appear at the top right the moment a technical word is spoken.
- **Camera** (`X.cam`) glides between framings, and the engine adds a slow drift so the picture is never frozen.
- **Characters** (`X.person`) breathe and blink, and can talk, wave and point. The same cast returns in every chapter: Ravi the customer in blue, the bank clerk in plum, the security officer, the attacker in a hood, vendor staff in teal.
- **Motion helpers** keep something moving at all times: `X.stream` (dots flowing along a path), `X.travel` (an object moving along a curve), `X.count` (a number counting up), `X.type` (typewriter text), `X.bob`, `X.pulse`, `X.shake`, `X.spin`.
- **Recap cards** (`X.recap`) dim the chapter and tick off the points as they are spoken.
- Subtitles, the chapter title and the progress bar (with a mark at each part) are drawn by the engine from `timing.js`, so they can never drift from the voice.

`GUIDE.md` in the source folder has the full rules for writing a chapter: the safe area (x 60–1860, y 130–930, above the subtitles), the allowed made-up identifiers, and the bugs to avoid (for example, never put a CSS animation and an SVG `transform` on the same element; wrap it in a group instead).

5. Rendering and building the files

`render.js START END 30 out.mp4` opens the player in headless Chromium with Playwright and, for every frame, calls `seek(frame / 30)`, takes a JPEG screenshot and pipes it into ffmpeg.

`build.py` runs the whole job:

1. `render`: cuts the 137,000 frames into 2.5-minute chunks along part boundaries and renders four at a time. A finished chunk is marked done, so a stopped run carries on where it left off.
2. `parts`: joins each part's chunks and compresses them once (H.264, `-tune animation`, which suits flat drawings), adds that part's slice of the voice, and joins the six parts into the full video with the whole voice track.
3. `srt`: writes subtitles for the full video and for each part.

GitHub refuses files over 100 MB, so the repository keeps the six part videos, and the single full video is kept outside it.

6. Try it: change one line

1. Edit the sentence in `script.txt`.
2. Run `python3 tts.py`. Only the changed line is recorded again; `timing.js` is rewritten.
3. Run `node snap.js check chapter:line:0.85` to save a picture of that moment and print any page errors.
4. Delete the `.ok` file of the chunks that cover that chapter in `out/chunks/`, then run `python3 build.py all`.

7. Checks before calling it done

- Snapshot every line at 85% of its length and part-way through long lines, and look at the contact sheets: nothing overlaps, nothing is cut off at the edge or hidden behind the subtitles.

- Every number on screen matches the script, and the script's maths is checked by running it (for example, the correct Verhoeff check digit of the token body 0982 6158 641 is 9, so a secret 3 gives the wrong last digit 2).
- The browser console shows no errors for any chapter.
- Each video's length matches `timing.json` within a frame, and the voice lines up with the subtitles at the start, the middle and the end.

Words used here

Word	Meaning
TTS	Text to speech: a program that turns written text into a voice.
ONNX	A file format for AI models that runs on many kinds of hardware without the original training code.
SVG	A drawing made of shapes described in text, so it stays sharp at any size.
Frame	One still picture. 30 frames per second makes smooth motion.
Headless browser	A real browser with no window, controlled by a script.
ffmpeg	A command-line tool that converts, joins and encodes audio and video.
H.264 / AAC	The most widely supported video and audio compression formats.
CRF	ffmpeg's quality dial for H.264: lower is sharper and bigger.
SRT	A plain-text subtitle file: number, start → end time, text.